

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence
PRN: 25UAM116
Name: Madhur Pravin Bhandari
Division: B
Date: 3 October 2026
GitHub Link: https://github.com/madhur-184/SLE-3_IAI

1. System Title & Short Description

System: 8-Puzzle Search Solver & Profiler. It is the same system I built in SLE-1 and profiled in SLE-2. It takes a scrambled 3×3 sliding-tile board and finds a sequence of blank-tile moves that reaches the goal, using Breadth-First Search (BFS) and depth-limited Depth-First Search (DFS, limit 30). While solving, it records the time taken and the number of nodes expanded. The problem it tackles is simple: show how uninformed search really behaves on a real state-space problem. In SLE-2, BFS always found the shortest paths (4, 12 and 18 moves) while DFS kept returning 30-move paths. This report draws the structure behind that code.

2. Context Diagram (Level 1)

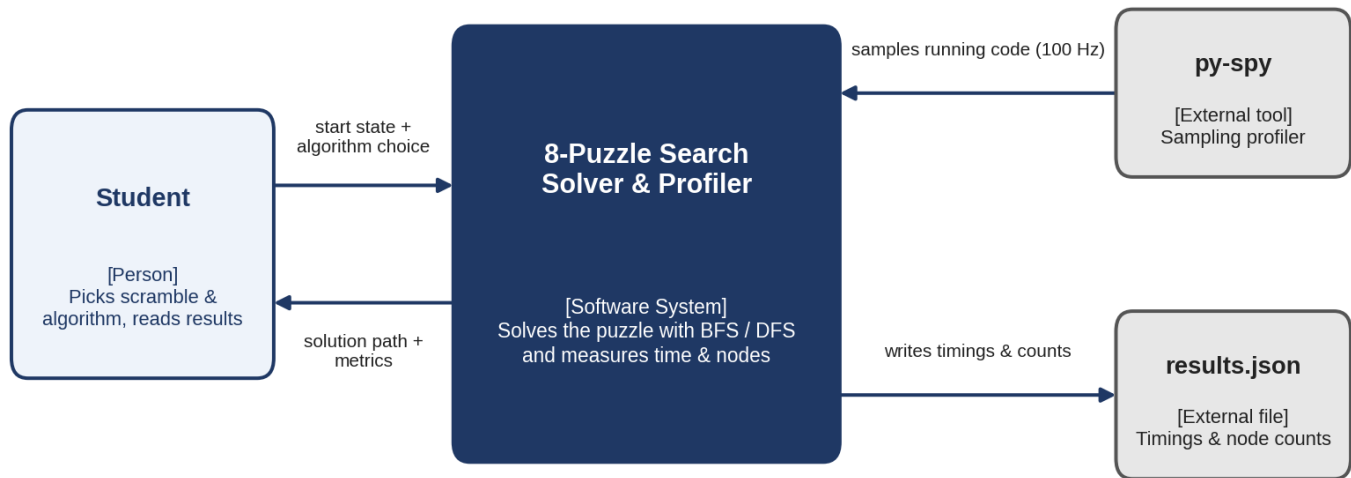


Fig. 1 – Level 1: System Context

The student is the only real user. They give the system a start state and pick BFS or DFS, and the system sends back the solution path along with its metrics. Two outside things touch the system: py-spy, which samples the program while it runs (100 samples per second), and results.json, where the timings and node counts are saved for the comparison tables in SLE-2.

3. Container Diagram (Level 2)

I split the system into six containers, shown in Fig. 2 below. Each one has a single job, which keeps the diagram easy to read and easy to explain:

- **Input Module:** takes the scrambled start state (the 4, 12 and 22-move test puzzles) and passes it on to the solver.
- **Problem Definition:** stores the board representation, the goal state and `get_neighbors()`, which produces the legal blank-tile moves.
- **Search Engine:** the core of the system. It runs `bfs()` or `dfs()` and decides the order in which states are explored.
- **Visited Set:** remembers states that were already explored so the search does not loop or repeat work.
- **Output Module:** reports the final path, its length and the number of nodes expanded.
- **Profiling Driver:** runs each algorithm 5 times on every test puzzle, measures the time with `time.perf_counter` and sends the numbers to the output.

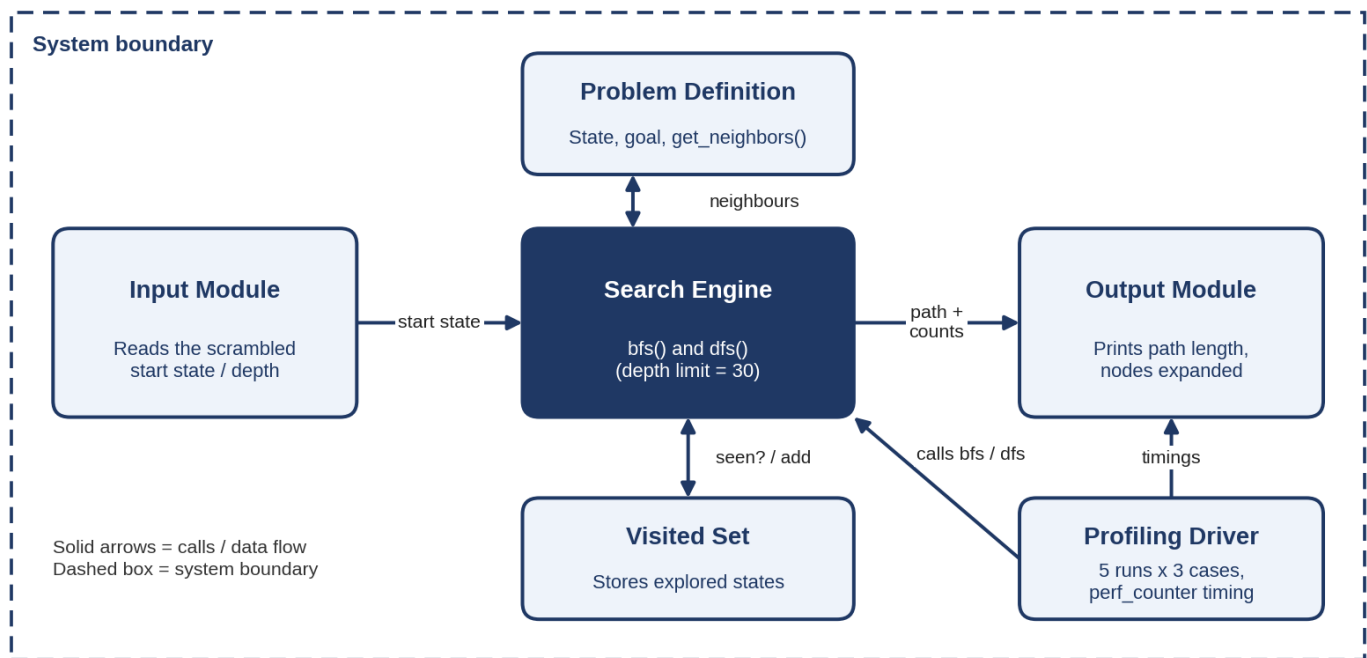


Fig. 2 – Level 2: Containers (6 main building blocks)

4. Component Diagram (Level 3) – Search Engine

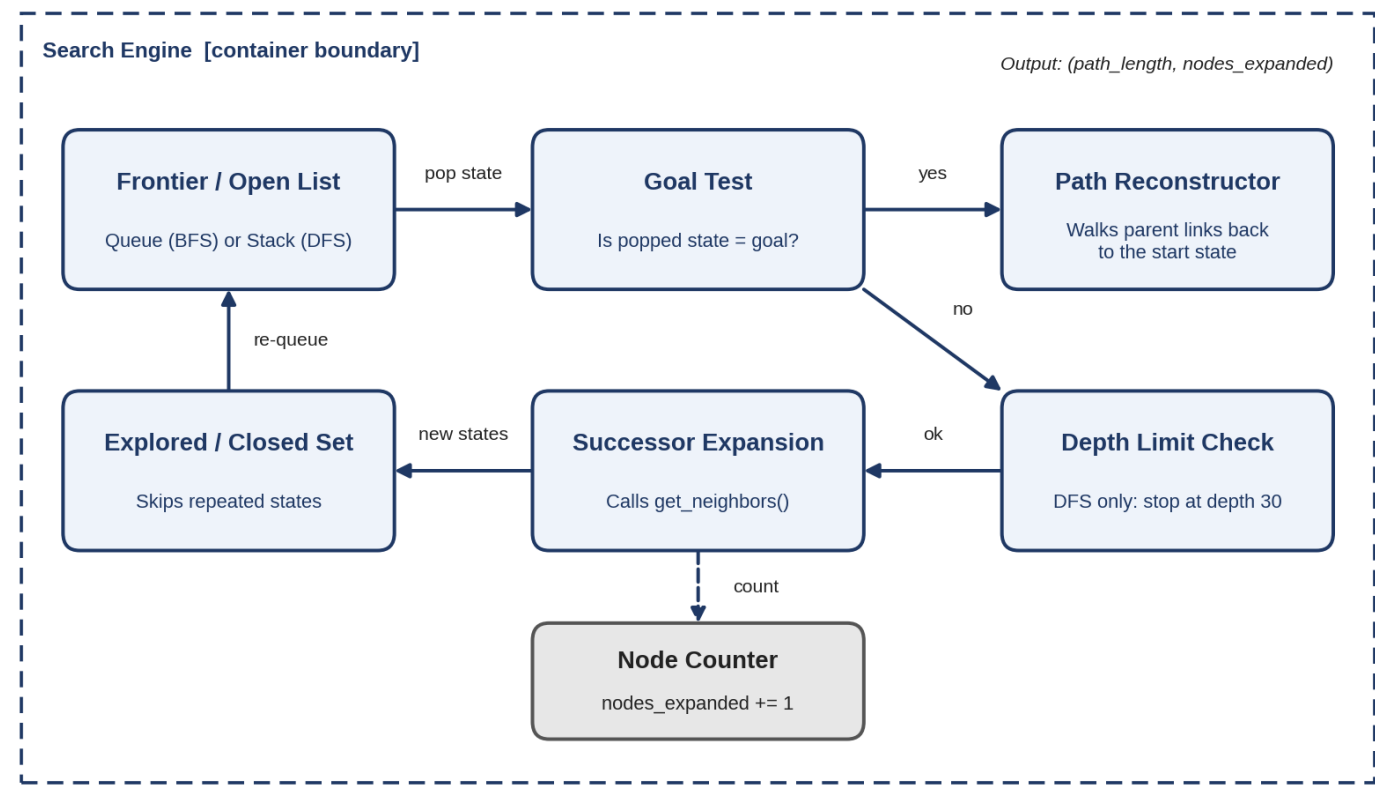


Fig. 3 – Level 3: Components inside the Search Engine container

I picked the Search Engine because it is the only container where BFS and DFS actually differ. The Frontier gives the next state to look at: a queue for BFS and a stack for DFS. The Goal Test checks that state. If it is the goal, the Path Reconstructor rebuilds the route by following parent links. If not, the Depth Limit Check (used only by DFS) decides whether to go deeper, and Successor Expansion calls `get_neighbors()` to create new states. The Explored Set filters out repeats before the remaining states go back to the Frontier. A small Node Counter ticks up each time a state is expanded.

5. Code Level Overview (Level 4)

Class / Function	One-line responsibility
<code>class State / Node</code>	Holds one board layout (0 = blank), its parent link and depth, so a path can be rebuilt later.
<code>GOAL_STATE, is_goal()</code>	The target layout and the check used by the goal test.
<code>get_neighbors(state)</code>	Returns every valid state reachable by sliding the blank up, down, left or right.
<code>bfs(start)</code>	Queue-based search; returns (path_length, nodes_expanded).
<code>dfs(start, limit=30)</code>	Stack-based, depth-limited search; returns the same (path_length, nodes_expanded).
<code>reconstruct_path(node)</code>	Follows parent links from the goal node back to the start state.

Class / Function	One-line responsibility
<code>run_experiments()</code>	Runs 5 timed runs on each of 3 scrambles with <code>time.perf_counter</code> and saves <code>results.json</code> .

How the four levels fit together:

C4 Level	Zoom	What is shown
1 – Context	Whole system	Student, py-spy, <code>results.json</code> (3 outside elements)
2 – Container	Inside the system	Input, Problem Definition, Search Engine, Visited Set, Output, Profiling Driver (6)
3 – Component	Search Engine only	Frontier, Goal Test, Path Reconstructor, Explored Set, Successor Expansion, Depth Limit Check, Node Counter (7)
4 – Code	Functions / classes	State / Node, <code>is_goal</code> , <code>get_neighbors</code> , <code>bfs</code> , <code>dfs</code> , <code>reconstruct_path</code> , <code>run_experiments</code> (7)

6. Design Decisions

I kept `get_neighbors()` in its own Problem Definition container so BFS and DFS share exactly the same move logic. That way, any difference in the results comes from the search strategy and nothing else. Both algorithms also return the same `(path_length, nodes_expanded)` pair, which made the SLE-2 comparison fair. The Profiling Driver sits outside the Search Engine so timing code never mixes with algorithm code. I stayed with 6 containers to keep the diagram readable, and a Heuristic Module could later be added as a 7th box if I extend this to A*.

7. AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** suggesting how to lay out the four C4 levels, drawing the first versions of the diagrams with a plotting script, and tidying the wording and Word formatting of this report.
- **What I did myself:** chose the 8-Puzzle system from SLE-1/SLE-2, decided how to split it into containers, picked the Search Engine for the component level, and checked that every box and function name matches my own code and my SLE-2 results.

8. Conclusion

Drawing the system at four levels showed me that architecture is mostly about deciding where to put boundaries. Context shows who and what the system talks to, Containers show the big parts, Components zoom into one part, and Code names the actual functions. Doing this for my own BFS/DFS project made it clear why they behave so differently: the structures are almost identical, and only the Frontier and the depth limit change. I also learned that separating the problem, the search and the profiling makes code easier to test and extend, for example when adding A* later.